Documentation Technique — CESIZen

Projet : CESIZen — L'application de votre santé mentale
Titre : Concepteur Développeur d'Applications (CDA)
Activité : 2 — Développer et tester les applications informatiques
Auteur : Anaïs
Date : Avril 2026

Sommaire

1. MCD — Modèle Conceptuel de Données
2. MLD — Modèle Logique de Données
3. Comparatif des solutions techniques
4. Choix final de la solution
5. Guide d'installation
6. Cahier de recette et scénarii de tests
7. Procédure de validation et modèle de PV de recette

---

1. MCD — Modèle Conceptuel de Données

Le Modèle Conceptuel de Données (MCD) représente les informations manipulées par l'application et les liens qui existent entre elles. C'est une vue qui permet de comprendre la structure des données avant de les implémenter en base de données.

Les entités

Le MCD de CESIZen est composé des entités suivantes :

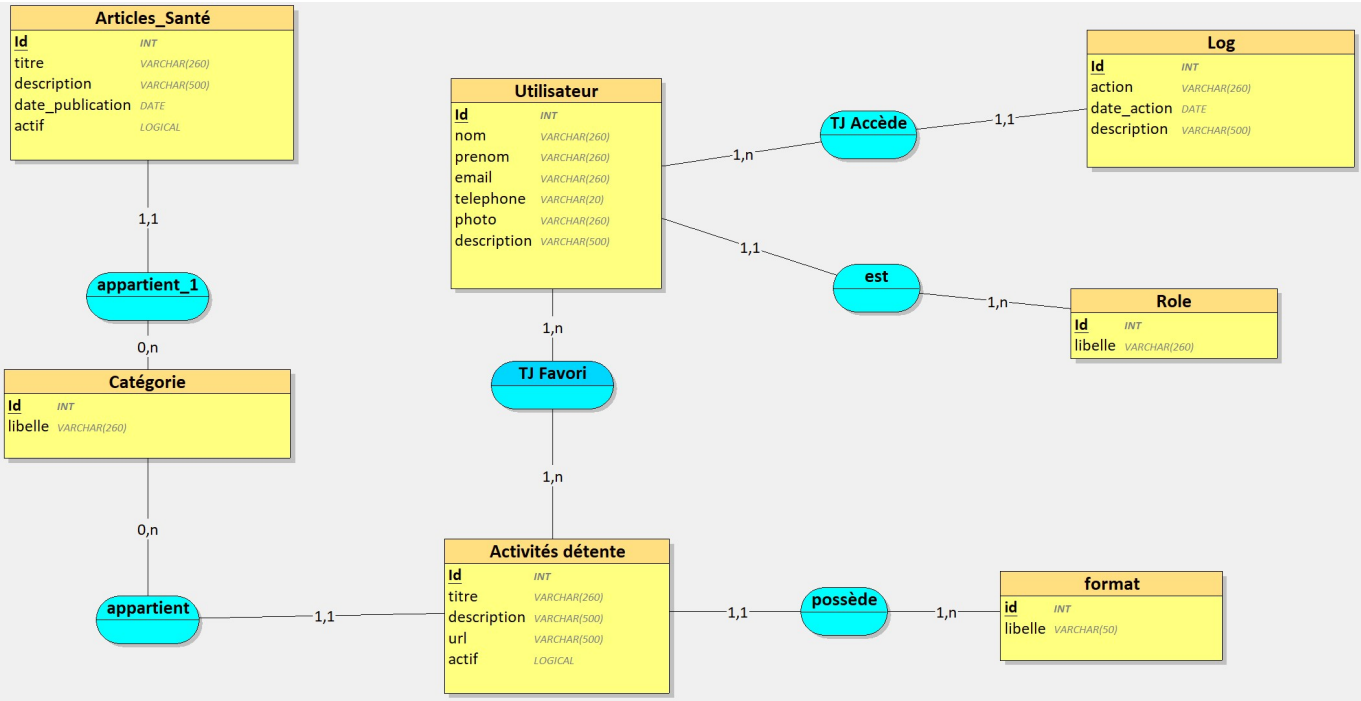
Entité	Description
Utilisateur	Toute personne ayant un compte sur l'application (utilisateur ou administrateur)
Role	Définit le niveau d'accès d'un utilisateur (utilisateur standard ou administrateur)
Articles_Santé	Articles informatifs sur la santé mentale, consultables par tous
Catégorie	Permet de classer les articles et les activités par thème
Activités_détente	Activités proposées aux utilisateurs pour gérer leur stress
Format	Définit le type de support d'une activité (vidéo, PDF, lien externe, etc.)
Log	Enregistre les actions réalisées par les administrateurs sur l'application

Les relations

Relation	Description

Relation	Description
est	Un utilisateur possède un rôle (utilisateur ou administrateur)
TJ_Accède	Un utilisateur peut générer des entrées dans les logs (actions d'administration)
TJ_Favori	Un utilisateur peut marquer plusieurs activités comme favorites
appartient_à	Un article appartient à une catégorie
appartient	Une activité appartient à une catégorie
possède	Une activité possède un format (vidéo, PDF, etc.)

Diagramme



---

2. MLD — Modèle Logique de Données

Le Modèle Logique de Données (MLD) est la traduction du MCD en tables concrètes, telles qu'elles sont réellement créées dans la base de données PostgreSQL. Chaque entité devient une table, et chaque relation devient une clé étrangère ou une table de liaison.

Les tables

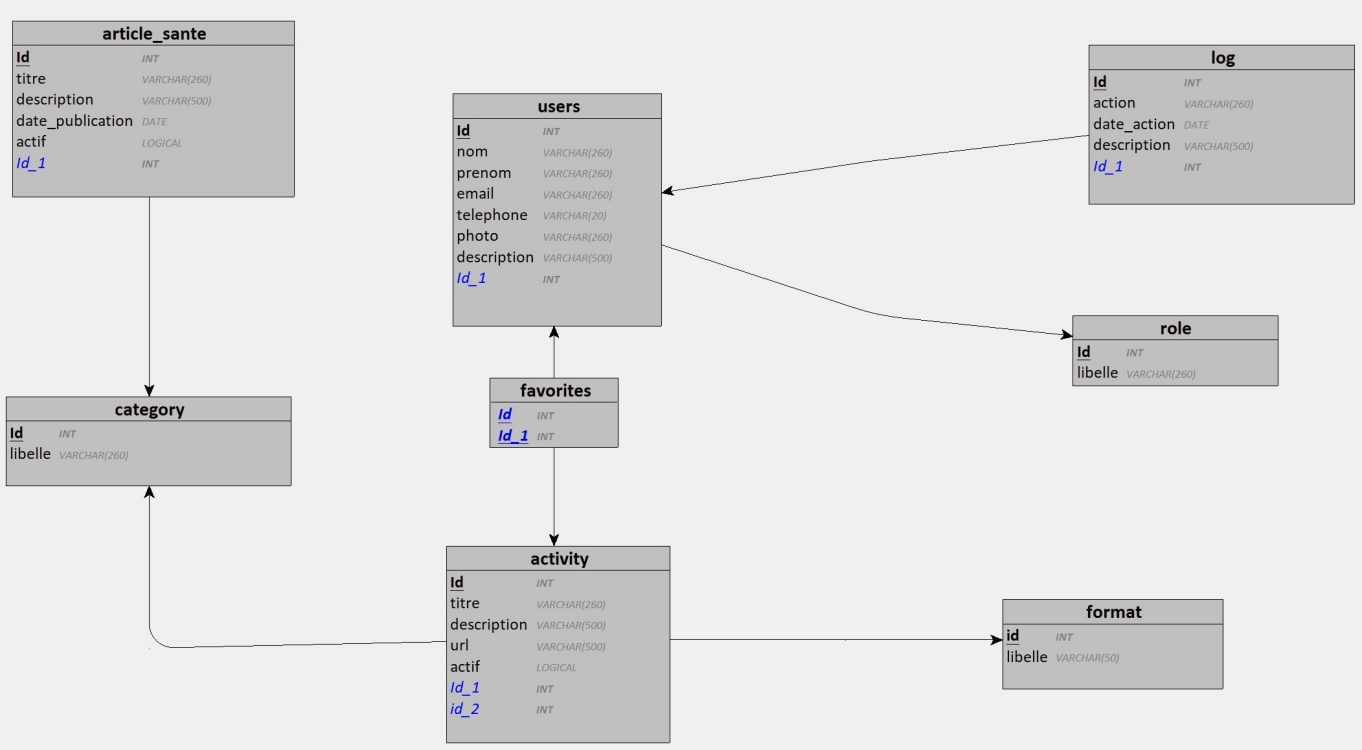
Table	Clé primaire	Clés étrangères	Description
utilisateur	`id`	`role_id` → role	Stocke les comptes utilisateurs
role	`id`	—	Contient les rôles disponibles (utilisateur, administrateur)
article_sante	`id`	`category_id` → category	Stocke les articles publiés

Table	Clé primaire	Clés étrangères	Description
category	`id`	—	Liste des catégories (articles et activités)
activity	`id`	`category_id` → category, `format_id` → format	Stocke les activités proposées
format	`id`	—	Types de supports disponibles pour les activités
favorites	`user_id` + `activity_id`	`user_id` → user, `activity_id` → activity	Table de liaison pour les favoris
log	`id`	`user_id` → user	Historique des actions des administrateurs

Points clés

- La table **favorites** est une table de liaison qui matérialise la relation plusieurs-à-plusieurs entre un utilisateur et ses activités favorites : un utilisateur peut avoir plusieurs favoris, et une activité peut être mise en favori par plusieurs utilisateurs. Elle n'a pas de clé primaire unique — c'est la combinaison de `user_id` et `activity_id` qui identifie chaque ligne.
- La table **log** est rattachée à un utilisateur via `user_id`, ce qui permet de savoir quel administrateur a effectué chaque action.
- Les tables **category** et **format** sont des tables de référence : elles contiennent des valeurs fixes qui sont réutilisées par d'autres tables.

Diagramme



3. Comparatif des solutions techniques

3.1 Contexte et objectifs

Avant de démarrer le développement de CESIZen, il a fallu choisir les technologies à utiliser. Ce choix est important car il conditionne la manière dont l'application sera construite, testée et maintenue dans le temps.

Les principales contraintes à prendre en compte étaient les suivantes :

- L'application doit être accessible à tout le monde (visiteurs non connectés, utilisateurs et administrateurs)
- Les données des utilisateurs doivent être protégées par une connexion sécurisée
- Un espace d'administration doit permettre de gérer les contenus et les comptes
- L'application doit communiquer via une API, c'est-à-dire un point d'échange entre le serveur et l'interface utilisateur
- Des tests automatisés doivent garantir que l'application fonctionne correctement à chaque modification

Trois solutions techniques ont été étudiées et comparées avant de retenir la solution finale.

3.2 Architectures étudiées

Architecture A — Django (Python) + Django Templates

Description :
Django est un framework Python très complet qui fournit tous les outils essentiels : gestion de la base de données, connexion des utilisateurs, interface d'administration, et génération des pages HTML directement côté serveur. Il n'est pas nécessaire d'ajouter de bibliothèques tierces pour démarrer un projet.

- **Côté serveur (backend) :** Django (Python)
- **Côté interface (frontend) :** Django Templates
- **Base de données :** PostgreSQL
- **Connexion des utilisateurs :** Système de sessions intégré à Django

Points forts :

- Très rapide à mettre en place grâce aux nombreux outils intégrés
- Framework stable, mature, avec une documentation très complète
- Sécurité robuste par défaut (protection contre les attaques courantes)

Points faibles :

- Les pages sont entièrement générées par le serveur : chaque changement à l'écran provoque un rechargement complet de la page, ce qui donne une expérience moins fluide
- La séparation entre l'interface et le serveur est peu marquée, ce qui rend l'évolution de l'un sans toucher à l'autre plus difficile
- Moins adapté si l'on souhaite proposer une application mobile ou un frontend indépendant dans le futur

---

Architecture B — FastAPI (Python) + Vue.js 3

Description :
FastAPI est un framework Python conçu pour créer des APIs. Vue.js 3 est un framework JavaScript qui permet de construire l'interface de l'application directement dans le navigateur de l'utilisateur, sans rechargement de page.

- **Côté serveur (backend) :** FastAPI (Python) + SQLAlchemy pour la gestion de la base de données
- **Côté interface (frontend) :** Vue.js 3
- **Base de données :** PostgreSQL
- **Connexion des utilisateurs :** JWT, un jeton de connexion sécurisé transmis à chaque requête

Points forts :

- Très performant : FastAPI est l'un des frameworks Python les plus rapides
- Une documentation de l'API est générée automatiquement et consultable en ligne depuis /docs
- L'interface Vue.js est fluide : la navigation ne recharge pas la page entière
- La séparation entre l'interface et le serveur est claire, ce qui facilite les tests et la maintenance

Points faibles :

- Nécessite de gérer deux projets distincts (backend et frontend), ce qui complexifie légèrement la mise en place initiale
- FastAPI est plus récent que Django : certaines fonctionnalités avancées nécessitent plus de configuration manuelle

---

Architecture C — Node.js (Express) + React

Description :
Express.js est un framework pour Node.js, qui permet de créer des APIs en JavaScript côté serveur. React est une bibliothèque développée par Meta (Facebook) pour construire des interfaces web en JavaScript côté navigateur.

- **Côté serveur (backend) :** Node.js + Express.js
- **Côté interface (frontend) :** React
- **Base de données :** PostgreSQL
- **Connexion des utilisateurs :** JWT

Points forts :

- JavaScript utilisé à la fois côté serveur et côté interface : un seul langage pour tout le projet
- React est la bibliothèque frontend la plus utilisée dans l'industrie : nombreuses ressources disponibles
- Interface fluide et réactive, sans rechargement de page

Points faibles :

- Express est minimaliste : il faut configurer manuellement beaucoup d'éléments (validation, sécurité, gestion des erreurs)
- La sécurité par défaut est moins complète que celle de Django ou FastAPI
- React a une courbe d'apprentissage plus difficile que Vue.js pour un projet de cette taille

3.3 Critères d'évaluation

Chaque architecture est notée de **1 à 5** pour chacun des critères ci-dessous. Un poids est attribué à chaque critère selon son importance dans le contexte du projet CESIZen.

#	Critère	Description	Poids
1	**Support et communauté**	Ancienneté de la technologie, taille de la communauté, qualité de la documentation	15%
2	**Rapidité de développement**	Facilité de prise en main, outils intégrés, gain de temps au démarrage	20%
3	**Performance**	Vitesse de réponse du serveur, capacité à gérer plusieurs utilisateurs simultanément	15%
4	**Sécurité**	Protections intégrées contre les attaques, gestion des rôles, chiffrement des mots de passe	15%
5	**Séparation interface / serveur**	Clarté de la séparation entre frontend et backend, facilité à faire évoluer chaque partie indépendamment	10%
6	**Testabilité**	Facilité à écrire et automatiser des tests, compatibilité avec les outils de CI/CD	15%
7	**Qualité de l'interface utilisateur**	Fluidité de navigation, réactivité, modernité de l'expérience côté utilisateur	10%

3.4 Tableau comparatif

Critère	Poids	Archi. A — Django + Templates	Archi. B — FastAPI + Vue.js 3	Archi. C — Express + React
Support et communauté	15%	**5**	**4**	**4**
Rapidité de développement	20%	**5**	**4**	**3**
Performance	15%	**3**	**5**	**4**
Sécurité	15%	**5**	**4**	**3**
Séparation interface / serveur	10%	**2**	**5**	**5**
Testabilité	15%	**4**	**5**	**4**
Qualité de l'interface utilisateur	10%	**2**	**5**	**5**
Score	**100%**	**3,75**	**4,55**	**3,85**

Classement final :

Position	Architecture	Score
1er	FastAPI + Vue.js 3	**4,55 / 5**
2ème	Express + React	**3,85 / 5**
3ème	Django + Templates	**3,75 / 5**

---

4. Choix final de la solution

4.1 Solution retenue

L'architecture retenue pour le projet CESIZen est l'**Architecture B : FastAPI (Python) + Vue.js 3**, avec PostgreSQL comme base de données et une authentification par JWT.

Cette solution obtient le meilleur score (**4,55 / 5**) et répond à l'ensemble des contraintes identifiées dans le cahier des charges.

4.2 Justification du choix

Côté serveur — FastAPI (Python)

FastAPI est un framework Python créé en 2018 qui permet de construire des APIs web de manière rapide et fiable. Il s'appuie sur des mécanismes du langage Python pour valider automatiquement les données reçues et garantir qu'elles correspondent bien à ce qui est attendu avant de les traiter ou de les enregistrer en base de données.

Pourquoi FastAPI pour CESIZen :

- **Rapidité :** FastAPI est l'un des frameworks Python les plus performants. Il peut traiter de nombreuses requêtes simultanément grâce à un mécanisme d'exécution asynchrone (les tâches n'attendent pas les unes les autres pour s'exécuter).
- **Validation automatique :** Grâce à la bibliothèque Pydantic, toutes les données envoyées à l'API sont vérifiées automatiquement. Cela réduit les risques d'erreurs ou d'injections de données malveillantes.
- **Documentation intégrée :** FastAPI génère automatiquement une page de documentation interactive (accessible sur `/docs`) qui liste toutes les routes disponibles et permet de les tester directement depuis le navigateur.
- **Gestion des droits d'accès :** Il est possible de définir facilement quelles routes sont accessibles aux visiteurs, aux utilisateurs connectés, ou uniquement aux administrateurs.
- **Gestion de la base de données :** L'intégration avec SQLAlchemy permet de manipuler la base de données en Python, sans écrire de requêtes SQL à la main.
- **Tests :** L'écosystème Python offre des outils de test très complets (`pytest`, `httpx`) qui s'intègrent bien avec FastAPI.

Côté interface — Vue.js 3

Vue.js 3 est un framework JavaScript qui permet de construire l'interface de l'application directement dans le navigateur. L'utilisateur navigue dans l'application sans que la page entière se recharge à chaque action.

Pourquoi Vue.js 3 pour CESIZen :

- **Prise en main accessible :** Vue.js est reconnu pour être plus simple à apprendre que React ou Angular, tout en restant très puissant pour des projets de taille moyenne.
- **Navigation fluide :** Grâce à Vue Router, le passage d'une page à l'autre est instantané côté navigateur, sans rechargement complet.
- **Gestion des données réactive :** Vue.js met automatiquement à jour l'interface dès qu'une donnée change (par exemple, l'affichage d'une liste de favoris après un clic).
- **Backend indépendant :** Le frontend communique avec l'API FastAPI uniquement via des requêtes HTTP. Les deux parties peuvent évoluer indépendamment.

Base de données — PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle fiable et performant, utilisé dans de nombreuses applications en production. Il garantit l'intégrité des données et s'intègre parfaitement avec SQLAlchemy. PGAdmin permet d'administrer facilement la base de données grâce à une interface graphique.

Authentification — JWT (JSON Web Tokens)

Lorsqu'un utilisateur se connecte, le serveur lui génère un jeton (token) qui contient ses informations de manière chiffrée. Ce jeton est ensuite envoyé avec chaque requête pour prouver que l'utilisateur est bien connecté. Cette approche est bien adaptée à une application où le frontend et le backend sont séparés.

4.3 Respect du pattern MVC

L'architecture retenue respecte le Design Pattern **MVC (Modèle — Vue — Contrôleur)**, qui consiste à séparer clairement les trois grandes responsabilités d'une application :

Couche	Rôle	Implémentation dans CESIZen
Modèle	Représente les données et leur structure	Modèles SQLAlchemy (`app/models/`) et schémas Pydantic (`app/schemas/`)
Vue	Ce que voit et utilise l'utilisateur	Composants Vue.js (`frontend/src/views/`)
Contrôleur	Traite les demandes et fait le lien entre les données et l'interface	Routes FastAPI (`app/controllers/`) et services métier (`app/services/`)

Cette séparation permet de modifier une partie de l'application sans risquer d'en casser une autre, et facilite grandement l'écriture des tests.

---

5. Guide d'installation

Ce guide explique comment installer et lancer l'application CESIZen sur votre ordinateur, étape par étape.

5.1 Prérequis

Avant de commencer, vérifiez que les logiciels suivants sont installés sur votre machine :

Logiciel	Version minimale	Utilité
Python	3.10 ou supérieur	Faire tourner le serveur FastAPI
Node.js	20 ou supérieur	Faire tourner l'interface Vue.js
PostgreSQL	14 ou supérieur	Base de données de l'application
Git	Toute version récente	Récupérer le code source

Pour vérifier qu'un logiciel est bien installé, ouvrez un terminal et tapez :

```
python --version
node --version
psql --version
git --version
```

Si l'une de ces commandes affiche une erreur, installez le logiciel manquant avant de continuer.

5.2 Récupérer le code source

Ouvrez un terminal et placez-vous dans le dossier où vous souhaitez installer le projet, puis exécutez :

```
git clone https://github.com/Anaderis/cesizen-python.git
cd cesizen-python
```

5.3 Créer la base de données

Ouvrez PostgreSQL (via pgAdmin ou en ligne de commande) et créez une base de données nommée `cesizen` :

```
CREATE DATABASE cesizen;
```

Ensuite, importez les données initiales grâce au fichier SQL fourni dans le projet :

```
psql -U postgres -d cesizen -f app/static/sql/cesizen-0104.sql
```

> Remplacez `postgres` par votre nom d'utilisateur PostgreSQL si celui-ci est différent.

5.4 Configurer la connexion à la base de données

Les informations sensibles (mot de passe, clé secrète) sont stockées dans un fichier `.env` qui n'est pas partagé sur GitHub pour des raisons de sécurité. Il faut le créer manuellement.

Un fichier modèle .env.example est fourni dans le projet. Copiez-le et renommez la copie `.env` :

Ouvrez ensuite le fichier `.env` et renseignez vos propres informations :

```
SECRET_KEY=remplacez_par_une_cle_secrete
DATABASE_URL=postgresql://nom_utilisateur:mot_de_passe@localhost:5432/cesizen
```

Remplacez `votre_utilisateur` et `votre_mot_de_passe` par vos identifiants PostgreSQL.

> Pour générer une clé secrète sécurisée, vous pouvez utiliser la commande suivante dans un terminal Python :
>
> ```
> import secrets; print(secrets.token_hex(32))
> ```

5.5 Installer et lancer le backend (FastAPI)

Étape 1 — Créer un environnement virtuel Python

Un environnement virtuel permet d'isoler les bibliothèques du projet sans affecter le reste de votre système.

```
python -m venv pythonCesizen
```

Étape 2 — Activer l'environnement virtuel

Sur Windows :

```
pythonCesizen\Scripts\activate
```

Activer l'environnement avec Powershell

```
.\pythonCesizen\Scripts\Activate.ps1
```

Sur Mac / Linux :

```
source pythonCesizen/bin/activate
```

Une fois activé, votre terminal affiche le nom de l'environnement au début de chaque ligne.

Étape 3 — Installer les dépendances

```
pip install -r requirements.txt
```

Sauvegarder les dépendances

```
pip freeze > requirements.txt
```

Étape 4 — Lancer le serveur

```
uvicorn app.main:app --reload
```

Le serveur est maintenant accessible à l'adresse : **http://localhost:8000**

La documentation interactive de l'API est disponible sur : **http://localhost:8000/docs**

5.6 Installer et lancer le frontend (Vue.js)

Ouvrez un **nouveau terminal** (gardez le premier ouvert pour le backend), puis placez-vous dans le dossier `frontend` :

```
cd frontend
```

Étape 1 — Installer les dépendances

```
npm install
```

Étape 2 — Lancer l'interface

```
npm run dev
```

L'interface est maintenant accessible à l'adresse : **http://localhost:5173**

5.7 Lancer les tests

Pour vérifier que tout fonctionne correctement, revenez dans le dossier racine du projet et exécutez :

```
pytest
```

Tous les tests doivent passer (affichage en vert). Si un test échoue, vérifiez que la base de données est bien configurée et que le serveur backend tourne correctement.

5.8 Résumé — ordre de démarrage

Pour utiliser CESIZen au quotidien, voici l'ordre à suivre à chaque démarrage :

1. Activer l'environnement virtuel Python (`pythonCesizen\Scripts\activate`)
2. Lancer le backend dans un terminal : `uvicorn app.main:app --reload`
3. Lancer le frontend dans un autre terminal : `cd frontend && npm run dev`
4. Ouvrir le navigateur sur **http://localhost:5173**

---

6. Cahier de recette et scénarii de tests

6.1 Contexte et périmètre

Ce cahier de recette décrit les tests réalisés pour valider le bon fonctionnement de l'application CESIZen. Il couvre les fonctionnalités suivantes :

Fonctionnalité	Description
Authentification	Connexion par email et mot de passe
Utilisateurs	Inscription, consultation, modification, désactivation, suppression
Articles de santé	Consultation publique, modification par l'administrateur
Activités bien-être	Consultation, création, modification, désactivation
Favoris	Ajout et suppression de favoris par un utilisateur connecté

6.2 Environnement de test

Les tests sont exécutés avec la commande `pytest` depuis le dossier racine du projet. Ils utilisent une base de données temporaire, distincte de la base de production, qui est créée au lancement des tests et supprimée à la fin.

Trois types de profils sont simulés pendant les tests :

Profil	Droits
Administrateur	Accès complet à toutes les fonctionnalités
Utilisateur connecté	Accès à son propre compte et à ses favoris
Visiteur (non connecté)	Accès aux contenus publics uniquement

---

6.3 Tests unitaires

Les tests unitaires vérifient des règles de validation précises, sans passer par l'application complète. Le responsable de tous les tests est le développeur back-end.

TU-01 — Validation du formulaire d'inscription

ID	Scénario	Résultat attendu
TU-01-01	Données valides	Compte créé sans erreur
TU-01-02	Email en majuscules	Email converti automatiquement en minuscules
TU-01-03	Email invalide	Erreur de validation
TU-01-04	Mot de passe trop court (< 8 caractères)	Erreur de validation
TU-01-05	Mot de passe sans majuscule	Erreur de validation
TU-01-06	Mot de passe sans chiffre	Erreur de validation
TU-01-07	Mot de passe sans caractère spécial	Erreur de validation
TU-01-08	Nom vide ou composé uniquement d'espaces	Erreur de validation
TU-01-09	Nom saisi avec des espaces autour	Espaces supprimés automatiquement

TU-02 — Validation de l'URL d'une activité

ID	Scénario	Résultat attendu
TU-02-01	URL YouTube complète	Acceptée
TU-02-02	Lien court youtu.be	Accepté
TU-02-03	Chemin vers un fichier PDF interne	Accepté
TU-02-04	Chemin vers un fichier audio interne	Accepté
TU-02-05	Champ URL vide	Accepté (champ optionnel)
TU-02-06	Texte quelconque sans format valide	Erreur de validation

TU-03 — Connexion utilisateur

ID	Scénario	Résultat attendu

ID	Scénario	Résultat attendu
TU-03-01	Email inconnu	Connexion refusée
TU-03-02	Mauvais mot de passe	Connexion refusée
TU-03-03	Identifiants corrects	Token de connexion retourné

TU-04 — Sécurité des mots de passe

ID	Scénario	Résultat attendu
TU-04-01	Création de compte avec mot de passe en clair	Mot de passe chiffré et illisible en base

6.4 Tests fonctionnels

Les tests fonctionnels vérifient le comportement des routes HTTP de l'API, notamment les codes de retour et les droits d'accès.

TF-01 — Authentification

ID	Scénario	Résultat attendu
TF-01-01	Connexion avec identifiants valides	200 — token retourné
TF-01-02	Connexion avec mauvais identifiants	401 — accès refusé
TF-01-03	Formulaire incomplet (email manquant)	422 — données incomplètes

TF-02 — Gestion des utilisateurs

ID	Scénario	Résultat attendu
TF-02-01	Consulter la liste (admin)	200 — liste retournée
TF-02-02	Consulter la liste (utilisateur)	403 — accès interdit
TF-02-03	Consulter la liste (visiteur)	401 — non authentifié
TF-02-04	Créer un compte (public)	200 — compte créé
TF-02-05	Créer un compte avec email invalide	422 — données invalides
TF-02-06	Créer un compte avec mot de passe faible	422 — données invalides
TF-02-07	Supprimer un compte (admin)	200 — compte supprimé
TF-02-08	Supprimer un compte (visiteur)	401 — non authentifié
TF-02-09	Modifier son propre compte	200 — compte mis à jour
TF-02-10	Modifier le compte d'un autre utilisateur	403 — accès interdit
TF-02-11	Modifier n'importe quel compte (admin)	200 — compte mis à jour

ID	Scénario	Résultat attendu
TF-02-12	Modifier un compte (visiteur)	401 — non authentifié

TF-03 — Désactivation de compte

ID	Scénario	Résultat attendu
TF-03-01	Désactiver son propre compte	200 — compte désactivé
TF-03-02	Désactiver le compte d'un autre	403 — accès interdit
TF-03-03	Désactiver n'importe quel compte (admin)	200 — compte désactivé
TF-03-04	Désactiver un compte (visiteur)	401 — non authentifié

TF-04 — Favoris

ID	Scénario	Résultat attendu
TF-04-01	Consulter ses favoris (connecté)	200 — liste retournée
TF-04-02	Consulter ses favoris (visiteur)	401 — non authentifié

TF-05 — Articles de santé

ID	Scénario	Résultat attendu
TF-05-01	Consulter la liste des articles (public)	200 — liste retournée
TF-05-02	Consulter un article (public)	200 — détail retourné
TF-05-03	Consulter un article inexistant	404 — non trouvé
TF-05-04	Modifier un article (admin)	200 — article mis à jour
TF-05-05	Modifier un article (utilisateur)	403 — accès interdit

TF-06 — Activités bien-être

ID	Scénario	Résultat attendu
TF-06-01	Consulter les activités (public)	200 — liste des activités actives
TF-06-02	Consulter toutes les activités (admin)	200 — liste complète
TF-06-03	Consulter toutes les activités (utilisateur)	403 — accès interdit
TF-06-04	Créer une activité (admin)	200 — activité créée
TF-06-05	Créer une activité (utilisateur)	403 — accès interdit
TF-06-06	Créer une activité (visiteur)	401 — non authentifié
TF-06-07	Désactiver une activité (admin)	200 — activité désactivée

ID	Scénario	Résultat attendu
TF-06-08	Modifier une activité (admin)	200 — activité mise à jour
TF-06-09	Modifier une activité (utilisateur)	403 — accès interdit
TF-06-10	Modifier une activité (visiteur)	401 — non authentifié
TF-06-11	Créer une activité avec URL invalide	422 — données invalides
TF-06-12	Modifier une activité avec URL invalide	422 — données invalides

6.5 Tests de non-régression

Les tests de non-régression vérifient que les règles du cahier des charges et les droits d'accès restent respectés après toute modification du code.

ID	Scénario	Résultat attendu
TNR-01-01	Tenter de créer un article via l'API	405 — action non autorisée
TNR-01-02	Tenter de supprimer un article via l'API	405 — action non autorisée
TNR-01-03	Tenter de supprimer une activité via l'API	405 — action non autorisée
TNR-02-01	Accéder à une page admin (utilisateur)	403 sur toutes les pages admin
TNR-02-02	Accéder à une page protégée (visiteur)	401 sur toutes les pages protégées
TNR-02-03	Modifier le compte d'une autre personne	403 — accès interdit

6.6 Récapitulatif

Catégorie	Nombre de cas
Tests unitaires	18 cas
Tests fonctionnels	30 cas
Tests de non-régression	6 cas
Total	**54 cas**

Tous les tests doivent passer sans erreur. Tout nouveau développement doit être accompagné d'au moins un test couvrant le cas nominal et un cas d'erreur.

---

7. Procédure de validation et modèle de PV de recette

7.1 Objectif

La procédure de validation permet de vérifier, avant toute mise en production, que l'application CESIZen fonctionne correctement et est conforme aux exigences définies dans le cahier des charges. Elle s'appuie sur les scénarios de tests décrits dans la section 6 et donne lieu à la rédaction d'un Procès-Verbal (PV) de recette signé par les parties concernées.

7.2 Étapes de la procédure

Étape 1 — Préparation

Avant de commencer les tests, s'assurer que :

- L'application est installée et fonctionnelle sur l'environnement de recette (voir section 5)
- La base de données est alimentée avec les données de test
- Les trois profils utilisateurs sont disponibles : administrateur, utilisateur connecté, visiteur

Étape 2 — Exécution des tests automatisés

Lancer la suite de tests automatisés depuis le dossier racine du projet :

```
pytest tests/ -v
```

- Tous les tests doivent passer (affichage en vert)
- En cas d'échec, noter le ou les tests concernés et corriger avant de poursuivre

Étape 3 — Rédaction du PV de recette

À l'issue des tests, compléter le modèle de PV de recette (section 7.3) avec les résultats obtenus, puis le faire signer par le responsable du projet.

Étape 5 — Décision

Résultat	Décision
Tous les tests conformes	Validation prononcée — mise en production autorisée
Anomalies mineures	Validation conditionnelle — corrections à apporter avant livraison
Anomalies bloquantes	Validation refusée — retour en développement

---

7.3 Modèle de PV de recette

PROCÈS-VERBAL DE RECETTE

Projet : CESIZen — Plateforme de bien-être mental
Version testée : _____
Date de recette : _____
Environnement : _____

Participants

Rôle	Nom	Signature

Rôle	Nom	Signature
Développeur		
Responsable du projet		

Résultats des tests automatisés

Résultat	Valeur
Nombre total de tests	
Tests réussis	
Tests échoués	
Commande exécutée	`pytest tests/ -v`

Résultats des tests par catégorie

Catégorie	Nb de cas	Conformes	Non conformes
Tests unitaires	18		
Tests fonctionnels	30		
Tests de non-régression	6		
Total	**54**		

Anomalies constatées

ID du test	Description de l'anomalie	Gravité (mineure / bloquante)	Statut

Gravité mineure : l'application fonctionne mais un comportement ne correspond pas exactement à l'attendu.
Gravité bloquante : une fonctionnalité essentielle ne fonctionne pas.

Décision

☐ Validation prononcée — mise en production autorisée
☐ Validation conditionnelle — corrections mineures à apporter
☐ Validation refusée — anomalies bloquantes à corriger

Commentaires :

Signatures

	Développeur	Responsable du projet

	Développeur	Responsable du projet
Nom		
Date		
Signature		